

Greedy Algorithms and Heaps

Greedy Algorithms

A greedy algorithm builds a solution piece by piece, always choosing the option that looks best at the current moment, without reconsidering earlier choices. Greedy is fast (usually $O(n \log n)$ from sorting) but only produces the correct/optimal answer for problems that have the 'greedy choice property' — not every problem is safely solvable greedily.

Classic Greedy Problems

- Activity Selection — choose the maximum number of non-overlapping activities by sorting by end time.
- Fractional Knapsack — take items by best value-to-weight ratio first (unlike 0/1 Knapsack, fractions are allowed here, which is why greedy works).
- Huffman Coding — build an optimal prefix code for data compression.
- Job Sequencing with Deadlines.

Heaps — What is a Heap

A heap is a complete binary tree that satisfies the heap property: in a Min-Heap, every parent is smaller than or equal to its children (so the smallest element is always at the root); in a Max-Heap, every parent is larger than or equal to its children. Heaps are the standard way to implement a Priority Queue.

Heap Time Complexity

- Get min/max (peek root): $O(1)$.
- Insert: $O(\log n)$.
- Delete min/max (extract root): $O(\log n)$.
- Build a heap from an unsorted array: $O(n)$.

Heaps in Python

```
import heapq

# Python's heapq module implements a Min-Heap by default

nums = [5, 1, 8, 3, 9]

heapq.heapify(nums)          #  $O(n)$ , rearranges in-place

heapq.heappush(nums, 2)      #  $O(\log n)$ 

smallest = heapq.heappop(nums) #  $O(\log n)$ , removes and returns smallest
```

```
# For a Max-Heap, negate values on push/pop
```

```
max_heap = []
```

```
for n in [5, 1, 8, 3]:
```

```
    heapq.heappush(max_heap, -n)
```

```
largest = -heapq.heappop(max_heap)
```

Common Interview Problems

- Kth Largest Element in an array using a min-heap of size k.
- Merge K Sorted Lists using a heap.
- Top K Frequent Elements.
- Find Median from a Data Stream using two heaps.
- Task Scheduler using a max-heap by frequency.